

Rapport Technique — Déploiement Ollama + Open WebUI sur Kubernetes (Minikube)

Environnement : Minikube (macOS, driver Docker), Namespace `llama-chat` **Stack :** Ollama, Open WebUI, NGINX Ingress Controller

1. Objectif du projet

Ce projet s'inscrit dans une démarche d'apprentissage des infrastructures Cloud-Native pour les applications d'intelligence artificielle. L'objectif est d'effectuer la migration d'une stack locale composée de deux services (**Ollama** et **Open WebUI**), initialement orchestrée via Docker Compose, vers un cluster Kubernetes local (**Minikube**).

Cette migration ne se limite pas à une conversion syntaxique de conteneurs : elle applique les pratiques de production Cloud-Native — isolation logique via Namespace, gestion dynamique du stockage persistant (PVC), contrôle fin de l'allocation des ressources CPU/RAM, sondes de santé (probes) à trois niveaux, et exposition contrôlée des services vers l'extérieur du cluster.

2. Architecture déployée



```

|           |
|           v           |
| Pod: ollama-deployment (image 0.33.2) |
|           |           |
| PVC open-webui-pvc (1Gi) <=> open-webui-deploy. |
| PVC ollama-pvc (10Gi->30Gi) <=> ollama-deploy. |
+-----+

```

3. Rôle de chaque ressource Kubernetes

Namespace (namespace.yaml)

Le Namespace `llama-chat` crée une frontière logique — et non physique — à l'intérieur du cluster, isolant l'ensemble des ressources du projet. Un cluster de développement mono-nœud pourrait techniquement s'en passer, mais son utilisation reste justifiée : c'est une exigence explicite du cahier des charges, elle évite toute collision de noms avec d'autres projets partageant le même cluster, elle réduit le *blast radius* (une seule commande, `kubectl delete namespace llama-chat`, supprime proprement l'ensemble du projet), et elle prépare le terrain pour une éventuelle logique multi-tenant avec quotas.

Deployment (ollama-deployment.yaml , open-webui-deployment.yaml)

Le Deployment pilote la boucle de réconciliation (*reconciliation loop*) de Kubernetes : il déclare un état désiré (nombre de replicas, image, configuration) et maintient cet état en continu.

Auto-guérison (self-healing) : contrairement à un Pod isolé (*naked pod*), la suppression ou le crash d'un Pod managé par un Deployment entraîne sa recreation immédiate par le `ReplicaSet` sous-jacent. Ce comportement a été vérifié concrètement en cours de projet : la suppression manuelle du Pod Ollama (`kubectl delete pod`) a entraîné sa recreation automatique en quelques secondes, sans intervention supplémentaire.

Service (ollama-service.yaml , open-webui-service.yaml)

Les Pods possèdent des adresses IP éphémères, réattribuées à chaque recreation, et ne disposent d'aucune résolution DNS par nom par défaut. Le Service fournit une abstraction réseau stable (IP virtuelle + nom DNS interne), qui route dynamiquement le trafic vers les Pods correspondant à son `selector`, quels que soient les changements d'IP sous-jacents.

- **ollama (ClusterIP)** — Service strictement interne au cluster, exposé sur le port `11434`. Seul Open WebUI a besoin de contacter Ollama ; aucun accès externe n'est nécessaire.

- **open-webui-service (NodePort)** — expose l'interface web sur le port `30080` du nœud, permettant un accès direct depuis un navigateur hors du cluster.

Distinction des trois ports d'un Service :

- **port** — port exposé par le Service à l'intérieur du cluster.
- **targetPort** — port réel sur lequel le conteneur écoute (doit correspondre à la vérité du conteneur, ici `11434` pour Ollama et `8080` pour Open WebUI).
- **nodePort** — port statique ouvert sur le nœud lui-même, accessible depuis l'extérieur (plage `30000–32767`).

ConfigMap (`configmap.yaml`)

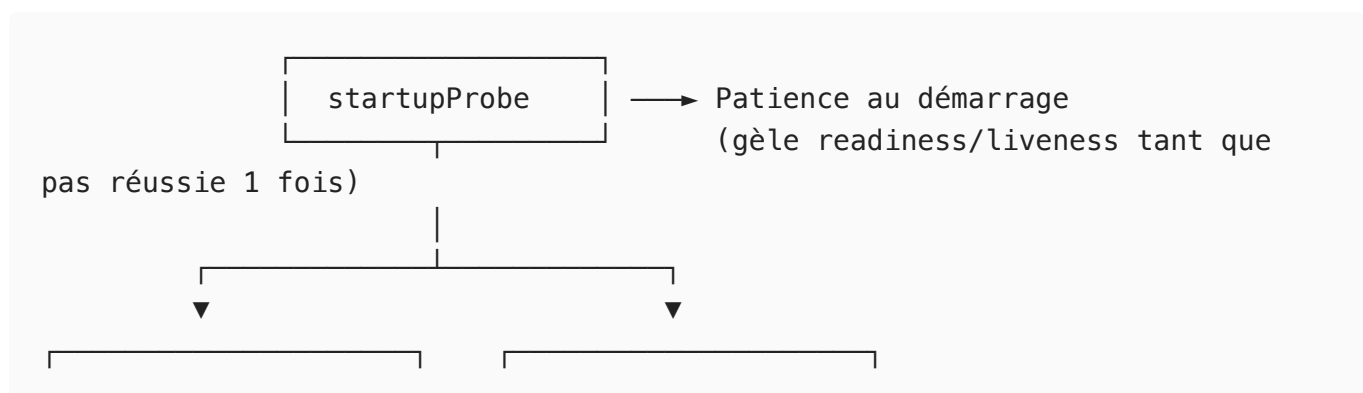
Le ConfigMap sépare la configuration applicative du code et des manifestes eux-mêmes, reproduisant en plus flexible le rôle du bloc `environment:` de Docker Compose. Il contient la variable `OLLAMA_BASE_URL`, injectée dans le conteneur Open WebUI, pointant vers le nom DNS complet (FQDN) du Service Ollama : `http://ollama.llama-chat.svc.cluster.local:11434`. Le choix du FQDN plutôt qu'un nom court a été fait délibérément, afin que la configuration reste valide même si l'architecture évoluait vers plusieurs namespaces.

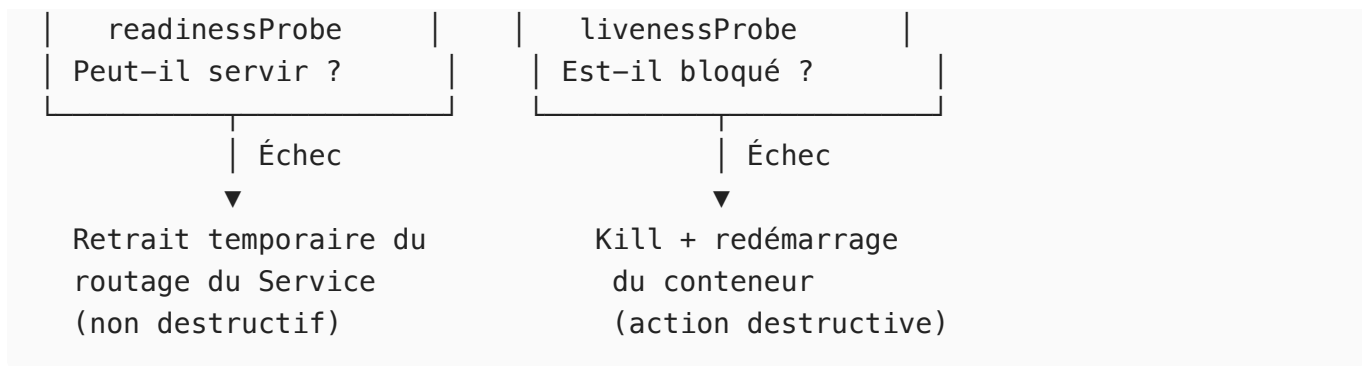
Persistent Volume Claim (`ollama-pvc.yaml` , `open-webui-pvc.yaml`)

Le système de fichiers d'un Pod est éphémère et disparaît avec lui. Le PVC est une demande de stockage externe qui survit indépendamment du cycle de vie des Pods, provisionnée dynamiquement via le `StorageClass` standard de Minikube (driver `hostpath`, qui s'appuie directement sur le disque de la machine hôte).

- **ollama-pvc** (10Gi initialement, redimensionné à 30Gi en cours de projet) — stocke les poids des modèles LLM dans `/root/.ollama`.
- **open-webui-pvc** (1Gi) — stocke la base de données SQLite et les données utilisateur dans `/app/backend/data`.

Probes (sondes de santé)





- **startupProbe** — accorde une patience prolongée au démarrage (essentiel pour Ollama, dont le chargement de modèles en environnement sans GPU peut prendre plusieurs minutes). Tant qu'elle n'a pas réussi une première fois, les deux autres probes sont suspendues — ce qui évite qu'un `livenessProbe` au timing strict ne tue prématurément un conteneur simplement lent à démarrer.
- **readinessProbe** — vérifie si le Pod peut traiter du trafic à l'instant présent. En cas d'échec, le Pod est retiré temporairement de la rotation du Service, sans être détruit.
- **livenessProbe** — détecte un blocage réel et durable. En cas d'échecs répétés, le conteneur est tué et redémarré.
- **Spécificité Ollama** : faute d'endpoint HTTP de santé exposé par l'image, les probes utilisent le mécanisme `exec` avec la commande `ollama list` — la même commande que celle utilisée dans le `healthcheck` du fichier Docker Compose d'origine.
- **Spécificité Open WebUI** : utilise le mécanisme `httpGet` sur l'endpoint `/health` (port `8080`), disponible nativement dans l'image.

4. Communication entre services

1. **Lecture de configuration** : le Pod Open WebUI lit la variable `OLLAMA_BASE_URL`, injectée depuis le ConfigMap.
2. **Résolution DNS** : cette valeur contient le nom FQDN `ollama.llama-chat.svc.cluster.local`, résolu par le DNS interne du cluster (CoreDNS) vers l'IP virtuelle du Service `ollama`.
3. **Routage** : le Service `ollama` redirige la requête, via son `selector` (`app: ollama`), vers le Pod Ollama actuellement en fonctionnement et validé `Ready` par sa `readinessProbe`.

5. Persistance des données et gestion des PVC

Sous Docker Compose, la persistance reposait sur des volumes nommés locaux. Sous Kubernetes, ce même besoin est couvert par l'abstraction **PV / PVC / StorageClass**, plus

explicite mais fonctionnellement équivalente.

Sans PVC, tout redémarrage du Pod Ollama entraînerait la perte intégrale des modèles déjà téléchargés (plusieurs gigaoctets chacun), nécessitant un nouveau `ollama pull` à chaque incident d'infrastructure. Ce constat a été vérifié concrètement : après un arrêt complet du cluster sur plusieurs jours (Minikube arrêté puis relancé), les données des PVC sont restées intactes, alors même que les Pods ont dû être recréés — démontrant que la persistance est bien découplée du cycle de vie des Pods.

Un redimensionnement dynamique du PVC Ollama (10Gi → 30Gi) a été réalisé en cours de projet pour accueillir simultanément les trois modèles testés (`llama3.2:3b` , `mistral:7b` , `gemma3:4b`), après activation de `allowVolumeExpansion: true` sur le `StorageClass`.

6. Comparaison : Docker Compose vs Kubernetes

Aspect	Docker Compose	Kubernetes (Minikube)
Paradigme	Impératif — démarre directement les conteneurs déclarés	Déclaratif — état désiré + boucle de réconciliation continue
Ordonnancement du démarrage	<code>depends_on</code> avec <code>condition: service_healthy</code>	Pas d'équivalent direct ; <code>readinessProbe</code> couvre partiellement ce besoin, mais après coup
Gestion des pannes	<code>restart: unless-stopped</code> , basique	Sondes à trois niveaux (startup, readiness, liveness) + reconciliation loop
Réseau & DNS	Réseau <code>bridge</code> personnalisé avec DNS embarqué Docker	Services K8s (ClusterIP, NodePort), DNS interne CoreDNS, Ingress
Configuration applicative	Bloc <code>environment</code> : codé en dur dans le fichier	<code>ConfigMap</code> séparé et référencé, réutilisable indépendamment
Stockage persistant	Volumes nommés Docker	<code>PersistentVolumeClaim</code> dynamique, adossé à une <code>StorageClass</code>
Isolation de projet	Réseau Docker dédié	Namespace
Auto-réparation	Redémarrage du même conteneur uniquement	Remplacement automatique par un nouveau Pod via <code>Deployment/ReplicaSet</code>

7. Choix techniques et justifications

- **Pinnage des images** à une version fixe (`ollama/ollama:0.33.2` , `ghcr.io/open-webui/open-webui:v0.11.3`) plutôt que des tags mouvants (`latest` , `main`), afin de garantir la reproductibilité du déploiement — un tag mouvant peut pointer vers une version différente à chaque pull, sans aucun avertissement.
 - **Dimensionnement des ressources** (`requests / limits`) basé sur l'analyse réelle de la capacité du nœud (`kubectl describe node`), plutôt que sur des valeurs arbitraires, afin d'éviter qu'un Pod reste bloqué `Pending` faute de ressources disponibles, ou soit `OOMKilled` en cours d'exécution.
 - **Stratégie d'exposition externe** : `NodePort` retenu en premier lieu pour sa simplicité en environnement de développement, complété par un **Ingress NGINX** pour un accès via nom de domaine (`llama.local`), plus proche des pratiques de production.
-

8. Incidents rencontrés et résolutions

1. **Confusion de contexte Kubernetes.** Une partie des premiers manifestes a été appliquée par erreur sur un cluster `kind` préexistant plutôt que sur Minikube, en raison du contexte `kubectl` actif à ce moment. `kubectl` ne signale jamais explicitement quel cluster il cible sauf consultation manuelle — résolu via `kubectl config get-contexts` et bascule vers le contexte `minikube`.
2. **Erreurs de syntaxe et de nommage.** `kind: PersistentVolumeClaim` mal orthographié, `apiVersion: app/v1` au lieu de `apps/v1`, noms d'objets non conformes aux règles DNS de Kubernetes (espaces, underscores interdits). Chaque erreur a été diagnostiquée directement via le message retourné par `kubectl apply`.
3. **Échecs de récupération d'image (`ImagePullBackOff`).** Diagnostiqués via `kubectl describe pod`, section `Events`: messages `i/o timeout` puis `context canceled`, révélant une connexion réseau instable plutôt qu'une erreur de configuration (tag ou nom d'image incorrects). Confirmé par un `docker pull` manuel en parallèle montrant la même lenteur.
4. **Redimensionnement de volume sur le driver `hostpath`.** Lors du passage de `ollama-pvc` de `10Gi` à `30Gi`, la colonne `CAPACITY` de `kubectl get pvc` n'affichait pas immédiatement la nouvelle valeur. Vérification faite que le driver `hostpath` de Minikube s'appuie directement sur le système de fichiers de la VM/hôte, sans partitionnement strict — le patch `allowVolumeExpansion: true` sur la `StorageClass` a permis le redimensionnement effectif.
5. **Incompatibilité modèle/fonctionnalité (`gemma3:4b`).** Le modèle a retourné l'erreur `does not support tools` lors de son utilisation dans Open WebUI. Diagnostiqué comme lié à

une fonctionnalité ("Tools"/appel de fonctions) activée côté interface, non supportée par ce modèle — et non à un problème d'infrastructure Kubernetes.

6. **Vérification en direct de l'auto-réparation.** La suppression manuelle du Pod Ollama (`kubectl delete pod`) a permis d'observer sa recreation automatique par le Deployment en quelques secondes, illustrant concrètement le principe de reconciliation loop.

9. Validation du déploiement

- **Pods** : `ollama-deployment` et `open-webui-deployment` à l'état `Running`, `READY 1/1`, sans `CrashLoopBackOff` ni `Pending` prolongé.
- **Services** : `ollama` (ClusterIP) et `open-webui-service` (NodePort) pleinement fonctionnels ; accès également validé via Ingress.
- **Modèles LLM** : trois modèles téléchargés dans le volume persistant d'Ollama et testés individuellement avec un prompt réel depuis l'interface :
 1. `llama3.2:3b`
 2. `mistral:7b`
 3. `gemma3:4b`
- **Accès** : interface accessible depuis un navigateur via NodePort (`minikube service --url`) et via Ingress (`http://llama.local`).

10. Bonus réalisés

- **Ingress NGINX** — activation de l'addon Ingress de Minikube, écriture et application du manifeste `ingress.yaml`, routage du nom d'hôte `llama.local` vers `open-webui-service`, configuration de `/etc/hosts` (`127.0.0.1 llama.local`) et ouverture d'un tunnel réseau (`minikube tunnel`) pour ponter le cluster vers macOS.
- **Redimensionnement dynamique de PVC** — activation de `allowVolumeExpansion: true` sur la StorageClass, permettant le passage de `ollama-pvc` de `10Gi` à `30Gi` sans perte de données.